

The GLP–Graph Architecture: Combining GLP–Mono with DMLG

Joachim De Witte

August 2026

©2026 Joachim De Witte. All rights reserved.

Published under the Creative Commons BY–NC–ND 4.0 license.

Abstract

The GLP–Graph (GLPG) architecture unifies two complementary paradigms of structured language generation: the deterministic graph-based governance of DMLG and the monolithic semantic engine of GLP–Mono. GLPG introduces a hybrid substrate in which grammar tokens, lemma transitions, paging geometry, and structural cadence are governed by an explicit canonical graph, while semantic content is injected through a single feedforward network with learned lemma embeddings. This synthesis resolves the limitations of both parent architectures: GLPG preserves DMLG’s deterministic routing, rule-based coherence, and PRE-driven structural validation, while inheriting GLP–Mono’s expressive lemma geometry, context-driven narrative memory, and stable next-token behaviour.

The resulting system is the first language architecture that operates simultaneously as a deterministic language graph and a fully emergent next-token model. GLPG supports two generative modes—paged graph traversal and pure MLP sampling—yielding a continuum between guided canonical prose and free-form semantic emergence. Training exhibits lifecycle-like behaviour, from proto-semantic attractors to mature canon compression, without overfitting or collapse. GLPG therefore constitutes a scalable, interpretable, and domain-stable substrate for hybrid intelligent systems, providing both structural guarantees and semantic flexibility within a single unified architecture.

Significance Statement

The GLP–Graph (GLPG) architecture represents a unification of two complementary paradigms of structured language generation: the monolithic semantic engine of GLP–Mono and the deterministic graph-based routing of DMLG. By integrating learned lemma geometry with explicit canonical transitions, GLPG provides a traceable, interpretable, and highly dynamic substrate for intelligent systems.

GLPG is architecturally significant for several reasons. First, it restores deterministic graph traversal as the primary mechanism for structural cadence, while retaining the expressive next-token behaviour of a single large MLP. Second, it offers a fully inspectable governance layer whose transitions, paging structure, and grammar rules remain explicit and auditable. Third, the architecture scales naturally within hybrid AI pipelines: GLPG supplies canonical structure and world geometry, while external models contribute stylistic refinement and surface fluency.

Beyond its engineering value, GLPG exhibits cognitive characteristics uncommon in conventional language models. Training proceeds through identifiable life-cycle phases, from proto-semantic attractors to mature canon compression, without collapse or overfitting. Even small datasets suffice to induce stable world models, making GLPG an efficient and domain-faithful compressor of narrative and semantic structure.

By combining the strengths of GLP–Mono (semantic absorption, expressive lemma geometry, recurrent narrative memory) with those of DMLG (deterministic routing, structural guarantees, PRE-based governance), GLPG reduces the limitations of both parent architectures and establishes a unified, scalable, and structurally grounded foundation for hybrid generative intelligence.

1 Introduction

Contemporary language models unify grammar, lexical content, and contextual structure into large, statistically entangled embedding spaces. While transformer-based architectures have achieved remarkable generative fluency, their internal organisation remains opaque: grammar is implicit, domain boundaries are soft, and long-horizon coherence depends on deep attention stacks rather than explicit structure. As a result, these systems provide limited guarantees about determinism, traceability, or domain fidelity—properties that become increasingly important as language models are deployed in hybrid intelligent systems.

Two recent architectures address these limitations from complementary directions. The Dynamic Modular Language Graph (DMLG) models language as deterministic graph traversal: tokens, transitions, and paging biomes form an explicit structural substrate, and generation emerges from context-sensitive navigation governed by rule-based coherence and PRE-driven sentence validation. In contrast, GLP-Mono demonstrates that coherent next-token behaviour can arise from a single monolithic MLP equipped with deterministic grammar tokens, paged lemma clusters, cosine-based ranking, and compact recurrent narrative memory. DMLG offers strong structural guarantees but limited semantic absorption; GLP-Mono offers expressive lemma geometry but weaker deterministic routing.

The GLP-Graph (GLPG) architecture introduced in this paper unifies these paradigms into a single hybrid system. GLPG retains DMLG’s deterministic graph routing, explicit paging, and rule-based grammar coherence, while inheriting GLP-Mono’s monolithic semantic engine, learned lemma embeddings, and stable next-token dynamics. This synthesis yields a substrate that is simultaneously deterministic and expressive: a language graph with emergent semantic behaviour, and a next-token model with explicit structural guarantees.

GLPG supports two generative modes. In paged graph traversal mode, deterministic routing restricts the target vocabulary per transition, enabling fast, stable, and canon-faithful generation. In pure MLP mode, the architecture evaluates the full vocabulary, allowing free-form semantic emergence similar to transformer models. The coexistence of these modes reveals that the monolithic network learns internal structure that mirrors the deterministic graph, while the graph provides a controllable manifold for semantic injection.

Training exhibits lifecycle-like behaviour: early models produce proto-semantic attractors, while sufficiently trained models compress entire narrative domains into stable canonical geometries. Even small datasets suffice to induce coherent world models, making GLPG an efficient canon compressor and a practical substrate for hybrid pipelines in which GLPG provides structure and external LLMs provide stylistic refinement.

By combining the strengths of DMLG and GLP-Mono while reducing their weaknesses, GLPG establishes a unified, scalable, and interpretable architecture for structured language generation. It provides deterministic routing, emergent semantics, and hybrid generative modes within a single system, forming a dynamic and domain-stable foundation for future hybrid intelligent systems.

2 From DMLG and GLP to GLPG

The GLP-Graph (GLPG) architecture emerges from the convergence of two structurally distinct yet complementary paradigms: the deterministic graph-based governance of DMLG and the monolithic next-token semantics of GLP-Mono. GLPG unifies these approaches into a single hybrid system that restores explicit structural determinism while preserving expressive lemma geometry and context-driven semantic behaviour.

DMLG models language as traversal through a canonical graph of grammar and lemma transitions. Its strengths lie in deterministic routing, strong structural guarantees, fast generation, and efficient learning. However, its reliance on numerous page-specific micro-models limits semantic absorption, reduces creative variation, and introduces training imbalance.

GLP-Mono, in contrast, demonstrates that coherent next-token behaviour can arise from a single

large MLP equipped with deterministic grammar tokens, paged lemma clusters, cosine ranking, and recurrent narrative memory. It offers strong semantic absorption and higher creative capacity, but its paging is weaker, its structural determinism is reduced, and both training and inference are slower.

GLPG combines the strengths of both architectures while reducing their weaknesses. It retains the monolithic semantic engine of GLP-Mono, enabling expressive lemma embeddings and stable next-token dynamics, but replaces learned page selection with deterministic graph-based routing derived from the last lemma token—a key feature of DMLG. This restores explicit structural cadence, improves paging determinism, and eliminates the need for page-specific micro-models. As a result, GLPG behaves once again as a language graph rather than a purely statistical language model.

A notable property of the architecture is its dual generative capability. The monolithic MLP can operate in paged mode, using deterministic graph routing to restrict the target vocabulary per transition, or in pure MLP mode, evaluating the full vocabulary without paging. The latter reveals that the network learns internal structure that mirrors the deterministic graph, enabling GLPG to function simultaneously as a structured language graph and as a fully emergent next-token model.

By integrating deterministic routing, explicit paging, rule-based grammar filtering, and monolithic semantic absorption, GLPG establishes a unified and scalable architecture that inherits the structural clarity of DMLG and the expressive power of GLP-Mono. It forms a dynamic, interpretable, and domain-stable substrate for hybrid generative systems.

3 GLP-Mono Improvements and DMLG Integration

GLPG extends the GLP-Mono architecture with a series of structural, contextual, and algorithmic improvements that restore deterministic graph routing while preserving the expressive semantic behaviour of the monolithic MLP. These modifications form the core of the GLPG architecture and reflect a synthesis of GLP-Mono innovations with the canonical graph principles of DMLG.

3.1 MLP Geometry and Activation

GLPG employs a deeper and more expressive monolithic network than GLP-Mono. The architecture uses four hidden layers instead of two, with the first hidden layer optionally configured to be significantly larger than subsequent layers. This increases the absorption capacity of the input context vector and improves semantic expressiveness.

To reduce computational cost and eliminate the instability associated with learned activation functions, GLPG replaces the learned activation MLP of GLP-Mono with a fixed SiLU activation. This prevents the “linear model trap” in which a single MLP collapses into quasi-linear behaviour, ensuring strong non-linear dynamics while substantially accelerating training and inference.

3.2 Context Encoding Enhancements

GLPG expands the context representation to provide richer structural signals to the monolithic network. In addition to the lemma-based current sentence embedding inherited from GLP-Mono, the model now incorporates:

- the current grammar sequence alongside the current lemma sequence,
- explicit embeddings for the previous grammar token and previous lemma token,
- a binary indicator for comma presence,
- a normalized line number to encode narrative progression.

These additions strengthen structural cadence, improve routing stability, and enhance long-horizon coherence.

3.3 Output Geometry and Embedding Dynamics

GLPG removes the page one-hot vector from the MLP output. Page selection is no longer learned but determined deterministically through graph routing based on the last lemma token, restoring the DMLG principle of canonical paging.

The lemma embedding update rule is simplified: instead of a gradually decaying damping schedule, GLPG uses a fixed small blending coefficient (e.g. $\alpha = 0.001$), improving stability and reducing training variance. Punctuation tokens are no longer implicitly present in all pages; they appear explicitly as graph transitions, increasing structural clarity.

3.4 Rule-Based Structural Filtering

GLPG integrates a deterministic rule-based filter that rejects incompatible grammar–lemma pairs before cosine ranking. This filter enforces:

- grammar compatibility constraints,
- lemma compatibility constraints,
- repetition suppression similar to the semantic engine.

This mechanism reduces invalid transitions and improves both structural coherence and sampling efficiency.

3.5 Paging Improvements

Paging is fully restored as a deterministic graph mechanism. GLPG uses the last lemma token as the routing key, with `EOL` treated as a special-case lemma. Pages may grow dynamically without affecting model size, though typical configurations limit the number of pages to 512 or 1024.

To prevent bloated pages, GLPG enforces a maximum number of inputs per page. Increasing the number of pages reduces the number of candidate tokens per transition, yielding faster generation, more deterministic behaviour, and fewer cosine evaluations.

3.6 Sampling Mechanism

Cosine distances produced by the monolithic MLP tend to be tightly clustered. Softmax sampling therefore produces nearly uniform probabilities across the top candidates. GLPG replaces softmax with a simpler normalization scheme:

$$p_i = \frac{(s_i - s_{\min} + \tau)}{\sum_j (s_j - s_{\min} + \tau)},$$

where τ is a small temperature increment ensuring non-zero probabilities.

To increase narrative decisiveness, GLPG applies top-5 boosting using Fibonacci scaling (e.g. 13, 8, 5, 3, 2), while retaining the remaining logits unchanged. Beam search becomes functional again due to deterministic graph routing, though it remains slower than paged sampling.

3.7 Training Regime

GLPG simplifies the training process by removing warm-up epochs entirely. Training proceeds exclusively through randomized epochs, with two context modes:

1. **Continuation mode:** each sequence is learned with the context of preceding sequences, producing long-form narrative behaviour.

2. **Replication mode:** each sequence is learned with its own context, producing story-to-story canonical compression.

All sample batches are preprocessed and cached once, as they remain constant throughout training. EOL transitions are deterministic and no longer learned.

3.8 Overall Result

These improvements yield substantially faster training and inference. The removal of learned activation, deterministic graph routing, and sample caching reduce computational overhead, while the expanded context and refined paging increase structural stability. GLPG exhibits generative capability at very low epoch counts and continues to refine semantic behaviour smoothly across extended training cycles.

4 Learning Behavior

GLPG exhibits a distinctive learning trajectory that mirrors several aspects of human language acquisition, despite being a fully artificial architecture. Training proceeds through recognisable developmental phases in which structural competence emerges early, followed by gradual semantic enrichment and world formation. This behaviour arises from the interaction between deterministic graph routing, monolithic semantic absorption, and the PRE-based recursive evaluation loop.

4.1 Early Structural Acquisition

During the initial epochs, GLPG learns grammar cadence and structural transitions extremely quickly. Deterministic graph routing provides a stable backbone that constrains generation from the outset, enabling the model to produce syntactically valid but semantically shallow sentences. This phase resembles a “child-like” linguistic stage: vocabulary is limited, semantics are minimal, and attractor-driven repetition dominates. The model already generates coherent structure, but meaning remains underdeveloped.

4.2 Gradual Semantic Enrichment

As training progresses, the monolithic MLP begins to inject semantic content into the deterministic structure. Cosine-based lemma selection allows context to modulate transitions within the graph, producing increasingly complex, coherent, and connected sentences. This phase corresponds to an “adult-like” linguistic stage: semantic fields expand, narrative relations strengthen, and world-specific motifs emerge. Structure remains stable due to graph routing, while semantics grow through repeated exposure to the curriculum.

4.3 Cognitive Parallels

Several components of GLPG exhibit functional analogues to cognitive processes:

- **Deterministic graph routing** simulates human incremental sentence construction, where grammar constrains allowable continuations.
- **The monolithic MLP** acts as a semantic injector, analogous to how humans integrate context and meaning into syntactic frames.
- **The PRE loop** (Propose–Retry–Evaluate) resembles recursive reasoning: proposing candidates, evaluating structural validity, and retrying when necessary.

These mechanisms collectively yield a form of artificial linguistic development that is structurally grounded yet semantically emergent.

4.4 Stability and Absence of Overtraining

GLPG shows no sensitivity to overtraining. Because paging, grammar transitions, and graph routing remain deterministic, the model cannot collapse into degenerate attractors. Instead, semantic refinement continues gradually across epochs, with improvements becoming smaller but consistently positive. The architecture supports early generative capability and maintains stable behaviour throughout extended training cycles.

4.5 Lifecycle Dynamics

Training in GLPG follows a lifecycle-like pattern:

1. **Proto-semantic phase:** rapid acquisition of grammar and structure, minimal semantics, strong attractors.
2. **Emergent semantic phase:** expansion of lemma geometry, improved coherence, formation of narrative motifs.
3. **Canon compression phase:** stable world models, domain-faithful generation, consistent structural and semantic behaviour.

This progression reflects the architecture’s ability to generate meaningful text early while continuing to refine its internal world geometry over time.

4.6 Training Dynamics

The average number of learned token transitions can be estimated as:

$$\text{learned transitions} = \frac{\text{total samples} \times \text{epochs}}{\text{number of sequences}}.$$

For example, with 200 sequences, 30,000 samples, and 200,000 epochs:

$$\frac{200,000 \times 30,000}{200} = 30 \text{ million}$$

estimated learned transitions. This illustrates how even small datasets, when cycled through many epochs, yield rich and stable canonical world models.

Overall, GLPG demonstrates a learning behaviour that is structurally deterministic, semantically emergent, and cognitively interpretable, forming a robust developmental trajectory uncommon in conventional language models.

5 Generation Modes: Graph Traversal and Pure MLP

GLPG supports two distinct generative modes that reflect its hybrid nature as both a deterministic language graph and an emergent next-token model. These modes—paged graph traversal and pure MLP sampling—provide complementary behaviours and reveal how the architecture integrates explicit structural determinism with learned semantic geometry.

5.1 Paged Graph Traversal Mode

In paged mode, GLPG uses deterministic graph routing to restrict the target vocabulary for each token transition. The last lemma token determines the active page, and only the grammar–lemma pairs within that page are considered during cosine ranking. This mechanism mirrors the canonical paging behaviour of DMLG and yields:

- fast generation due to reduced candidate sets,
- high structural determinism,
- strong canon stability,
- guided narrative flow,
- explicit manifold structure.

Rule-based grammar and lemma filtering further reduce invalid transitions, ensuring tight structural coherence. Paged mode therefore behaves as a deterministic language graph with semantic modulation provided by the monolithic MLP.

5.2 Pure MLP Mode

In pure MLP mode, GLPG evaluates the entire target vocabulary for each token transition. Paging is disabled, and the model relies solely on cosine similarity between the predicted lemma embedding and all stored output embeddings. This mode removes structural determinism and exposes the learned internal geometry of the monolithic network.

Pure MLP mode reveals that the network has implicitly learned manifold structure from the training data: semantic zones, attractors, and narrative tendencies emerge without explicit graph constraints. The behaviour resembles small transformer models:

- diffuse semantic organisation,
- emergent world models,
- conceptual and drifting narrative flow,
- self-balancing semantic emergence.

However, rule-based grammar and lemma filtering remain active, preventing invalid transitions and maintaining minimal structural coherence.

Pure MLP mode is computationally expensive: cosine evaluation over the full vocabulary is typically an order of magnitude slower than paged traversal. A practical acceleration trick is to sample a random subset of 512–1024 tokens per step and include the period token to ensure sentence completion. This preserves variation while significantly improving speed.

5.3 Hybrid Nature of GLPG

The coexistence of these two modes demonstrates that GLPG is the first architecture to combine a deterministic language graph with a fully emergent next-token model. Paged mode shows explicit structure; pure MLP mode shows implicit structure learned by the network. Together, they reveal that GLPG’s monolithic MLP internalises aspects of the deterministic graph, while the graph provides a controllable manifold for semantic injection.

5.4 Comparison of Generative Modes

Paged traversal provides deterministic structure with semantic modulation; pure MLP mode provides emergent semantics with minimal structural constraints. GLPG’s ability to operate in both regimes is central to its identity as a hybrid architecture that unifies explicit canonical structure with learned semantic behaviour.

Property	Paged Mode	Pure MLP Mode
Generation Style	Structured prose	Associative cloud
Semantic Organization	Clustered, zone-based	Diffuse, transformer-like
Canon Stability	High, explicit	Moderate, emergent
Narrative Flow	Linear, guided	Conceptual, drifting
Manifold Structure	Explicit (paging)	Implicit, self-learned
Grammar Behavior	Deterministic and tight	Deterministic but looser
World Model	Organized and coherent	Emergent and broad
Overall Feel	Guided narration	Free-form thought
Cognitive Character	Structured reasoning	Self-balancing emergence

6 Formal Definition of GLPG

A GLP–Graph (GLPG) is a hybrid, structurally deterministic, and semantically emergent language generator built on top of a canonical grammar–lemma graph and a monolithic semantic engine. It unifies the explicit structural substrate of DMLG with the learned lemma geometry of GLP–Mono. This section provides a formal definition of the GLPG architecture.

6.1 Canonical Graph: Structural Substrate

GLPG is grounded in a canonical grammar–lemma graph

$$G = (V, E, P),$$

where:

- V is the set of canonical tokens, consisting of grammar tokens and lemma tokens;
- $E \subseteq V \times V$ is the set of admissible transitions, defined by grammar rules, paging structure, and domain-specific constraints;
- $P = \{p_1, p_2, \dots, p_n\}$ is the set of pages, each representing a compact grammar–lemma biome.

The graph is *canon-closed*: no unseen tokens or transitions may be generated. Grammar tokens define deterministic structural invariants; lemma tokens define semantic content.

6.2 Deterministic Paging via Last Lemma Routing

Paging is determined by a deterministic routing function

$$\pi : V \rightarrow P,$$

which maps the last lemma token to its corresponding page. The special token EOL acts as a dedicated routing key. Paging restricts traversal to a small subset of grammar–lemma pairs, reducing search complexity and enforcing domain structure.

Unlike GLP–Mono, page selection is not learned. It is derived entirely from the canonical graph, restoring the deterministic routing behaviour of DMLG.

6.3 Monolithic Semantic Engine

GLPG employs a single feedforward network

$$f(x) = W_4(\sigma(W_3(\sigma(W_2(\sigma(W_1x))))))$$

with four hidden layers and fixed SiLU activation σ . The network outputs:

- a normalized lemma embedding $\hat{e} \in \mathbb{R}^d$,
- a structural context vector used for ranking candidate transitions.

Lemma embeddings for grammar–lemma pairs are stored as $e_{G,L}$ and updated through a fixed blending rule:

$$e_{G,L} \leftarrow (1 - \alpha)e_{G,L} + \alpha\hat{e},$$

with small constant α (e.g. 0.001). This yields stable long-term geometry.

6.4 Context Compression and Structural Signals

The input vector x consists of:

- compressed sentence history,
- recurrent narrative memory,
- current lemma and grammar sequences,
- previous lemma and grammar tokens,
- punctuation indicators,
- normalized line number,
- story-level sequence embedding.

These signals provide both structural cadence and semantic context.

6.5 Rule-Based Coherence Layer

GLPG enforces deterministic coherence through a rule-based filter

$$\Phi : V \times V \rightarrow \{0, 1\},$$

which rejects transitions violating:

- grammar adjacency rules,
- lemma compatibility constraints,
- repetition suppression criteria.

This filter operates before cosine ranking and ensures structural validity.

6.6 Transition Scoring and Selection

GLPG selects the next grammar–lemma pair through a multi-stage pipeline integrating deterministic structural filtering with learned semantic geometry. The process consists of: candidate selection, cosine scoring, ranking, normalized sampling, and optional boosting.

1. Candidate Selection Let $\mathcal{V}$ be the full vocabulary of grammar–lemma pairs.

In paged mode, The candidate set $\mathcal{C}$ is defined as:

$$\mathcal{C}_{\text{paged}} = \Phi(P_{\pi(\ell_{\text{last}})}).$$

In pure MLP mode, the candidate set is:

$$\mathcal{C}_{\text{mlp}} = \Phi(\mathcal{V}).$$

In accelerated MLP mode, the candidate set is:

$$\mathcal{C}_{\text{acc}} = \Phi(\mathcal{V}_{\text{subset}}).$$

where:

- $P_{\pi(\ell_{\text{last}})}$ is the page selected by deterministic last-lemma routing,
- $\mathcal{V}_{\text{subset}}$ is a random subset of 512–1024 pairs,
- $\Phi(\cdot)$ is the rule-based filter removing grammar-incompatible, lemma-incompatible, or repetitive transitions.

This ensures that only structurally valid transitions are considered.

2. Cosine Scoring For each candidate $(G, L) \in \mathcal{C}$, GLPG computes:

$$s(G, L) = \cos(\hat{e}, e_{G,L}),$$

where $\hat{e}$ is the predicted lemma embedding and $e_{G,L}$ is the stored embedding.

3. Ranking and Top- k Selection Candidates are sorted by descending score:

$$(G_1, L_1), (G_2, L_2), \dots, (G_k, L_k),$$

and truncated to the top- k items (typically $k = 20$). Let $s_i = s(G_i, L_i)$ denote the sorted scores.

4. Normalized Sampling GLPG uses linear normalization with temperature increment τ :

$$p_i = \frac{s_i - s_{\min} + \tau}{\sum_{j=1}^k (s_j - s_{\min} + \tau)},$$

where $s_{\min}$ is the minimum score in the top- k set.

5. Integrated Top-5 Boosting To increase narrative decisiveness, GLPG applies a boosting vector:

$$B = [13, 8, 5, 3, 2, 1, \dots, 1],$$

yielding boosted probabilities:

$$p_i^{\text{boost}} = \frac{B_i \cdot (s_i - s_{\min} + \tau)}{\sum_{j=1}^k B_j \cdot (s_j - s_{\min} + \tau)}.$$

Sampling is performed over p_i^{boost} .

Summary The transition scoring pipeline integrates:

- deterministic structural filtering,
- graph-based paging or full-vocab evaluation,
- cosine-based semantic scoring,
- top- k ranking,
- temperature-stabilized normalization,
- Fibonacci-style boosting.

This yields a next-token mechanism that is both structurally coherent and semantically expressive.

6.7 Sentence-Level PRE Governance

GLPG incorporates a sentence-level Propose–Retry–Evaluate (PRE) mechanism:

1. **Propose**: generate a candidate sentence via paged or pure MLP mode;
2. **Evaluate**: apply deterministic structural checks (grammar validity, closure, repetition suppression);
3. **Retry**: regenerate the sentence if evaluation fails.

PRE ensures global structural integrity independent of token-level sampling.

6.8 Dual Generative Modes

GLPG supports two generative regimes:

- **Paged Graph Traversal**: deterministic routing restricts candidate tokens, yielding structured, canon-faithful prose;
- **Pure MLP Mode**: the full vocabulary is evaluated, yielding emergent, transformer-like semantic behaviour.

Both modes share grammar determinism and rule-based filtering.

6.9 Summary

A GLPG is defined by:

- a deterministic canonical grammar–lemma graph,
- lemma-based paging with explicit routing,
- a monolithic semantic engine with learned lemma geometry,
- rule-based structural coherence,
- PRE-based sentence governance,
- and dual generative modes integrating structure and emergence.

GLPG is therefore a hybrid architecture: a deterministic language graph with an emergent semantic manifold, forming a unified substrate for structured and expressive language generation.

7 Example Model Size and Parameters

To illustrate the compact geometry of a typical GLPG model, we provide a concrete example based on a mid-scale configuration. Despite its expressive behaviour, the model remains small: the persisted Python model occupies only 40–50 MB on disk, including all learned parameters, lemma embeddings, paging structures, and metadata.

7.1 Network Geometry

The monolithic MLP uses four hidden layers with the following sizes:

- first hidden layer: 1536 units,
- subsequent hidden layers: 768 units each,
- lemma output embedding dimension: 128,
- total pages: 512 (minimal influence on model size),
- maximum page input size: 16 (no influence on model size).

7.2 Input Vector Composition

The input context vector is composed of several structural and semantic components:

$$\begin{aligned}\text{compressed sentence history} &= 24 \times 5 + 12 \times 15 = 300, \\ \text{narrative memory} &= 128, \\ \text{current grammar + lemma sentence} &= 2 \times (30 \times 2 + 1) = 122, \\ \text{previous grammar + lemma + comma flag} &= 2 \times 16 + 1 = 31, \\ \text{sequence embedding} &= 8, \\ \text{line number} &= 1.\end{aligned}$$

Thus the total input vector dimension is:

$$590 = 300 + 128 + 122 + 31 + 8 + 1.$$

7.3 Parameter Count

The total number of learned parameters in the monolithic MLP is:

$$\text{MLP parameters} = 590 \times 1536 + 1536 \times 768 + 768 \times 768 + 768 \times 768 + 768 \times 128 \approx 3,366,608.$$

This includes all weights and biases across the four hidden layers and the output layer.

7.4 Vocabulary and Embedding Storage

Each grammar–lemma pair has a learned output embedding of dimension 128. Thus:

$$1\text{M parameters} \approx 8000 \text{ vocabulary entries.}$$

For example:

- a vocabulary of 6000 pairs requires $\approx 0.75\text{M}$ parameters,
- a vocabulary of 8000 pairs requires $\approx 1.0\text{M}$ parameters.

Paging structures (512 pages) store only text-based routing metadata and have negligible impact on model size.

7.5 Total Model Size

Combining:

- $\approx 3.37\text{M}$ MLP parameters,
- $\approx 0.5\text{--}1.0\text{M}$ lemma embedding parameters,
- paging metadata and routing tables,

yields a persisted Python model of approximately:

40–50 MB.

This demonstrates that GLPG achieves expressive semantic behaviour and deterministic graph routing within a compact parameter budget, enabling fast training, fast inference, and efficient deployment.

8 Example Datasets

To illustrate the behaviour of GLPG across domains, we evaluate four datasets of varying size, structure, and narrative density. All experiments use the same model geometry described in the previous section: a 40–50 MB persisted Python model with a 590-dimensional input vector, four hidden layers, 128-dimensional lemma embeddings, and 512 pages.

Dataset size refers to raw text size prior to tokenization. Model size refers to the stored Python model, not the parameter count. Vocabulary size denotes the number of distinct grammar–lemma pairs discovered during training. The number of samples corresponds to the unique token transitions learned by the model; this number is extremely small compared to what transformer models typically require for functional behaviour.

8.1 Dataset Overview

Dataset	Raw Size	Sequences	Samples	Vocab
forest (LLM)	20 KB	25	3,432	586
jeekyll & hyde (Gutenberg)	139 KB	201	26,706	4,285
odyssey (Gutenberg)	611 KB	806	121,144	7,716
planet (self-recursive)	1,418 KB	720	254,108	5,489

Table 1: Dataset characteristics and learned vocabulary sizes for a fixed GLPG model geometry.

Despite the large variation in dataset size, the stored model size remains between 39 MB and 50 MB. This demonstrates that GLPG scales primarily with vocabulary size and not with raw dataset length, enabling compact world engines even for large narrative corpora.

8.2 Training Behaviour: 3K vs 200K Epochs

To study learning dynamics, we compare undertrained models (3,000 epochs) with sufficiently trained models (200,000 epochs). All experiments use identical model geometry and identical datasets.

3K Epochs: Child-Like Proto-Language At 3K epochs, GLPG exhibits a “child phase” characterised by:

- rapid acquisition of grammar and structure,
- minimal semantic absorption,

- strong attractor behaviour around a few dominant lemmas,
- repetitive pseudo-grammar,
- absence of global coherence.

This phase does *not* resemble human child language. A more accurate analogy is:

A child who already knows words but has no grammar, and who obsessively repeats one semantic theme.

Undertrained models produce artificial proto-language: semantically narrow, structurally rudimentary, and dominated by attractor kernels. This behaviour is similar to:

- tiny transformers trained on a single corpus,
- distilled GPT-2 variants,
- attractor networks,
- emergent multi-agent languages,
- minimal world models.

The model compresses the dataset into a small set of semantic attractors but cannot yet reconstruct full syntax or narrative structure.

200K Epochs: Adult-Like Canon Compression At 200K epochs, GLPG transitions into a mature “adult phase”:

- stable syntactic behaviour,
- domain-wide vocabulary usage,
- coherent narrative flow,
- fully developed canonical world geometry,
- attractor *worlds* instead of attractor *kernels*.

This stage resembles:

- NanoGPT trained on a single book,
- Char-transformers trained on one corpus,
- distilled world-models,
- story-world agents,
- mini-LLMs with domain training.

The model reconstructs a complete semantic world with stable grammar and rich domain motifs, but without general-purpose language competence. This is an artificial linguistic stage with no human analogue: a fully developed world engine with narrow domain focus.

8.3 Comparison of Learning Stages

Property	Undertrained (3K)	Sufficiently Trained (200K)
Syntax	fragmentary	stable, variable
Vocabulary	very limited	domain-wide
Canon	rudimentary	fully developed
Coherence	local	partially global
Variation	low	moderate
Semantics	attractor core	attractor world

Table 2: Comparison between early-stage and mature GLPG training behaviour.

8.4 Interpretation

Across all datasets, GLPG demonstrates a consistent developmental trajectory:

1. **Proto-semantic phase:** rapid structural learning, minimal semantics.
2. **Emergent semantic phase:** expansion of lemma geometry, improved coherence.
3. **Canon compression phase:** stable world models and domain-faithful generation.

The evolution from 3K to 200K epochs corresponds to the transition from artificial proto-language to mature domain-language: a form of linguistic development unique to structured neural systems and not observed in human language acquisition. Remarkably, the same GLPG model geometry can absorb datasets with widely varying canon, scale, and narrative density—ranging from miniature corpora such as *forest*, to full-length Gutenberg novels such as *hyde* and *odyssey*, and even fully self-recursive synthetic datasets such as *planet*. This demonstrates that GLPG does not merely learn a dataset, but constructs a stable canonical world model regardless of dataset size. The architecture compresses each domain into a coherent semantic manifold while preserving structural determinism, enabling a single fixed-size model to function effectively across heterogeneous narrative worlds.

9 Micro-Model with Very Small Dataset

One of the defining strengths of DMLG was its ability to produce coherent, domain-faithful output even when trained on extremely small datasets. GLPG restores this capability: the combination of deterministic graph routing, compact lemma geometry, and rule-based structural filtering allows even microscopic models to generate recognizable literary worlds.

9.1 Model Geometry

For this experiment, we use a drastically reduced GLPG configuration:

- first hidden layer: 144 units,
- subsequent hidden layers: 72 units,
- lemma output dimension: 32,
- total pages: 128,
- max page input size: 16.

The total number of learned MLP parameters is:

$$590 \times 144 + 144 \times 72 + 72 \times 72 + 72 \times 72 + 72 \times 32 \approx 118,000.$$

Vocabulary embeddings contribute:

$$385 \times 32 \approx 12,000,$$

yielding a total of roughly 130,000 parameters plus page metadata. The persisted Python model occupies only ≈ 1.5 MB.

9.2 Dataset Characteristics

The micro-model is trained on a tiny corpus:

- dataset size: 6 KB raw text,
- sequences: 3,
- samples: 994 unique token transitions,
- vocabulary: 385 grammar–lemma pairs.

Despite the minimal dataset, GLPG constructs a stable canonical world model. Structure, canon, and repetition alone suffice to induce a coherent literary domain.

9.3 Paged vs Pure MLP Generation

We evaluate the micro-model at two training stages—1K epochs and 20K epochs—and in two generation modes: paged graph traversal and pure MLP sampling.

1K Epochs (Child Phase) At 1K epochs, both paged and pure MLP modes produce artificial proto-language:

- strong attractor behaviour around a few dominant lemmas,
- repetitive pseudo-grammar,
- narrow semantic field,
- minimal narrative coherence.

Paged mode yields more structural stability, while pure MLP mode produces diffuse, transformer-like drift. In both cases, the model already exhibits a recognizable “observatory world”: ridges, domes, stars, instruments, and the caretaker appear consistently, despite the microscopic dataset.

20K Epochs (Adult Phase) At 20K epochs, the micro-model transitions into mature canon compression:

- stable syntax and page routing,
- coherent narrative motifs,
- domain-wide vocabulary usage,
- emergent world geometry,

- consistent structural cadence.

Paged mode produces structured prose with clear canonical transitions. Pure MLP mode produces free-form associative text that nevertheless remains faithful to the observatory domain. The model reconstructs a miniature literary world with only 130K parameters.

9.4 Interpretation

This experiment demonstrates that GLPG can compress even a tiny dataset into a coherent semantic manifold. The architecture does not require large corpora: deterministic graph routing, rule-based filtering, and compact lemma geometry allow a fixed-size model to function across radically different dataset scales—from miniature corpora such as *observatory*, to full-length Gutenberg novels, to fully self-recursive synthetic datasets.

GLPG therefore restores a key property of DMLG: the ability to generate structured, domain-faithful text even at microscopic parameter counts. The micro-model behaves as a complete world engine, despite being smaller than many classical RNNs or tiny transformers.

10 Conclusion

GLPG demonstrates that structured language generation does not require vast parameter counts, transformer-scale datasets, or opaque attention mechanisms. By unifying the deterministic graph routing of DMLG with the monolithic semantic engine of GLP-Mono, the architecture establishes a hybrid paradigm in which explicit structure and emergent semantics reinforce rather than constrain one another. Deterministic paging, rule-based coherence, compact lemma geometry, and dual generative modes together form a stable substrate capable of producing coherent, domain-faithful text across a wide range of training regimes.

The learning behaviour of GLPG follows a clear developmental trajectory: rapid acquisition of grammar and structural cadence, gradual semantic enrichment, and eventual formation of a mature canonical world model. This progression is robust across dataset sizes and domains. A single fixed-size GLPG model can absorb miniature corpora such as *forest*, full-length Gutenberg novels such as *jeekyll & hyde* and *odyssey*, and even fully self-recursive synthetic datasets such as *planet*. In each case, the architecture compresses the domain into a coherent semantic manifold while preserving structural determinism, enabling consistent generation without architectural changes or scale-dependent tuning.

The micro-model experiment further illustrates the resilience of the architecture. With only 130K parameters and fewer than one thousand training samples, GLPG reconstructs a recognizable literary world, complete with stable motifs, structural cadence, and emergent semantic attractors. This capability, once unique to DMLG, is restored and strengthened in GLPG: even microscopic models behave as complete world engines.

Together, these results show that GLPG provides a compact, interpretable, and scalable foundation for structured language generation. It bridges the gap between deterministic language graphs and emergent neural semantics, offering a unified architecture that remains functional at every scale—from tiny micro-models to large domain compressors. GLPG demonstrates that meaningful world formation can arise from structure, repetition, and canonical constraints alone, and that hybrid architectures can achieve expressive behaviour without sacrificing determinism, efficiency, or clarity.

A Related Work

A.1 Deterministic Language Graphs and Structured Routing

Early graph-based language systems modelled text generation as traversal through explicit state machines or grammar graphs. Classical approaches include deterministic finite automata, probabilistic context-free grammars, and rule-driven dialogue systems. These models provided strong structural guarantees but limited semantic absorption.

DMLG extended this lineage by introducing paged lemma biomes, rule-based coherence, and PRE-driven sentence validation. Its deterministic routing and explicit canonical structure enabled fast generation and high interpretability, though semantic expressiveness was constrained by its reliance on numerous page-specific micro-models.

GLPG inherits the structural clarity of DMLG while resolving its semantic limitations through monolithic lemma geometry.

A.2 Monolithic Next-Token Models

Single-network next-token architectures have been explored in various forms, including early feedforward language models, RNN-based sequence generators, and compact transformer variants such as NanoGPT-style single-book models. These systems demonstrate that coherent next-token behaviour can emerge from relatively small networks when trained on domain-specific corpora.

GLP-Mono belongs to this lineage: it showed that a single MLP equipped with grammar tokens, paged lemma clusters, cosine ranking, and recurrent narrative memory can produce stable next-token behaviour without attention mechanisms. However, its learned paging reduced determinism and weakened structural guarantees.

GLPG restores deterministic routing while retaining monolithic semantic absorption.

A.3 Hybrid and Modular Generative Architectures

Hybrid architectures that combine explicit structure with learned semantics have appeared in several domains. Examples include:

- neural-symbolic systems that integrate rule-based reasoning with learned embeddings,
- modular world-model pipelines where symbolic planners interact with neural generative components,
- graph-neural architectures that embed structured transitions into learned manifolds,
- domain-compressed mini-transformers trained on single books or narrow corpora.

These systems aim to balance interpretability with expressive power. GLPG contributes a distinctive variant of this paradigm: a deterministic language graph with an emergent semantic manifold learned by a single feedforward network.

A.4 Domain Compression and World Models

Recent work on world models, distilled transformers, and single-corpus LLMs shows that small architectures can compress entire narrative domains into compact latent geometries. NanoGPT-style models trained on one book, tiny transformers trained on narrow corpora, and distilled GPT-2 variants all demonstrate domain-faithful generation at small scales.

GLPG extends this idea by providing explicit structural guarantees. Its deterministic paging, rule-based filtering, and canonical transitions ensure that domain compression remains stable, interpretable, and resistant to collapse—even in micro-models with fewer than 200K parameters.

A.5 Positioning GLPG

GLPG occupies a unique position within the landscape of generative architectures:

- like DMLG, it provides deterministic routing, explicit paging, and rule-based structural coherence;
- like GLP–Mono, it provides expressive lemma geometry, semantic absorption, and stable next-token behaviour;
- like hybrid neural-symbolic systems, it unifies explicit canonical structure with emergent neural semantics;
- like domain-compressed world models, it produces coherent literary worlds even at microscopic parameter counts.

This combination yields a hybrid architecture that is simultaneously deterministic and emergent, interpretable and expressive, compact and domain-stable. GLPG therefore contributes a structurally grounded alternative to transformer-based language models, particularly in settings where determinism, traceability, and canonical world formation are essential.

B Triadic Cosmos Ecosystem Context

The present paper is part of the broader *Triadic Cosmos* ecosystem: a unified research program that develops a compositional ontology of intelligence together with modular, inspectable computational architectures. The ecosystem spans conceptual theory, executable reference implementations, and fully documented empirical demonstrations, forming a coherent framework for studying hybrid intelligent systems.

All materials, extended notes, and evolving documentation are publicly available in the open GitHub project:

<https://github.com/triadic-cosmos>

Within the `triadic-system` repository, a complete reference implementation of the GLPG engine, the narrator pipeline, and all supporting modules is provided under the AGPL-3.0 license.

B.1 Scope and Ecosystem Boundaries

Scaling GLP–Graph to the size of contemporary LLMs—billions of parameters, massive datasets, and industrial training budgets—is outside the scope of the ecosystem. The ecosystem is built around unification, structural clarity, and compact world engines rather than large-scale pretraining. GLP–Graph is designed to be small, deterministic, and domain-stable, providing a canonical substrate for hybrid generative systems rather than serving as a standalone large LM.

References

- [1] J. De Witte, “Grammar Lemma Paged Language Models,” Triadic Cosmos Ecosystem, 2026.
- [2] J. De Witte, “Learnable Lemma Embeddings for GLP Models,” Triadic Cosmos Ecosystem, 2026.
- [3] J. De Witte, “The GLP–Mono Architecture,” Triadic Cosmos Ecosystem, 2026.
- [4] J. De Witte, “Language as Graph Traversal: The DMLG Architecture,” Triadic Cosmos Ecosystem, 2026.
- [5] J. De Witte, “A Dynamic and Modular Architecture for Language Modeling,” Triadic Cosmos Ecosystem, 2026.